TatvaSoft

SOFTWARE DEVELOPMENT · GUIDELINE

TVS-DEV-AI-GL-01.2

Context Engineering

Context engineering is simply the habit of giving the AI only the exact information it needs for the job right now—nothing more, nothing less.

VERSION	EFFECTIVE DATE	PREPARED BY	APPROVED BY
1.0	28.09.2026	Vishal Shah	Shital Patel

HOW TO READ THIS DOCUMENT

Short on time? Read the **Key Takeaway** in the Introduction, then skim the **Do's** and **Don'ts** headings. Each Do and Don't has its own section with a short explanation.

Parent guideline: **TVS-DEV-AI-GL-01 AI Development Tools Guidelines** (Context Engineering section).

Document Control

Document Control Information

Document Title	Context Engineering
Document ID	TVS-DEV-AI-GL-01.2
Document Version	Version 1.0
Effective Date	28.09.2026
Parent Document	TVS-DEV-AI-GL-01 — AI Development Tools Guidelines

Document Publication History

Document Prepared By	Vishal Shah
Document Reviewed By	Parthesh Mehta
Document Approved By	Shital Patel
Initial Release Version	1.0
Initial Release Date	28.09.2026

Document Revision History

Version No.	Revision Description	Change Author	Change Approver	Revision Date
1.0	Final Release	Vishal Shah	Shital Patel	25.09.2026

Document Definition

Context engineering is simply the habit of giving the AI only the exact information it needs for the job right now—nothing more, nothing less. Here are a few simple Do’s and Don’ts to help you get better, faster results from your AI editor.

Contents

Introduction	Why managing context matters
Do’s	5 practices to follow
Don’ts	4 practices to avoid

Introduction

> **KEY TAKEAWAY**
>
> Context engineering is simply the habit of giving the AI **only the exact information it needs** for the job right now—nothing more, nothing less.

Working with AI coding tools isn't just about crafting the perfect prompt—it's mostly about **managing what the AI can see** at any given moment.

HOW THE AI READS CONTEXT

AI models don't "think" like humans do; they read the **entire chat history and every attached file** on **every single turn**.

WHEN CONTEXT GETS CLUTTERED

When that space gets cluttered with **old code, failed attempts, or off-topic chatter**, the AI gets confused and starts making mistakes.

Here are a few simple Do's and Don'ts to help you get better, faster results from your AI editor.

Do’s

Start a fresh chat when switching tasks or when debugging gets noisy

- Treat AI conversations as **temporary space for a single task**.
- If you’ve finished a feature, or if three bug fixes in a row have failed, **clear the slate**.
- Carry forward only the **core problem** and the **latest error message** into a new thread.

Point to files instead of repasting them

- If your AI editor already has indexing or file access, refer to files **by path** or `@mention` them directly.
- Save your prompt space for explaining *what* you want changed and *why*.

Lock in project rules using persistent configurations

- Use `.cursorrules`, system prompts, or workspace settings for guidelines that **never change**—like code style, stack preferences, or linting requirements.

→ *This keeps your daily prompts short and focused.*

Document decisions in your repo, not in chat history

- When an experiment works out or a key architecture decision is made, **write it down in your README or docs**.

→ *Chat logs disappear; repository documentation lasts.*

Define strict boundaries before unleashing AI Agents

- Before letting an AI agent run, **set clear boundaries**.

Tell it:

What it's allowed to edit	What it shouldn't touch	What a successful fix actually looks like

Don’ts

Don’t let dead-end experiments pollute your active conversation

- Once an approach fails or gets rejected, **don't keep dragging old, failed ideas through a long chat**.



Don’t dump the entire codebase into the prompt

- Throwing too much code at the AI just **confuses it**, making it more likely to give **wrong answers** or **break untouched files**.

Don’t force-feed specific tools or skills if the AI can figure it out

- Avoid over-specifying every micro-step or forcing custom skills **unless a task genuinely demands a non-standard workflow**.
- **Trust the editor** to select the right capability when the goal is clear.

Don’t rely on AI chat as your project’s single source of truth

- Never depend on previous conversation turns to remember **business logic or complex requirements**.
- If a rule matters long-term, **commit it to your codebase or docs**.

— End of Document —